Generalized Einsum with Row Variables: Shape Inference for Deep Learning in OCaml

LUKASZ STAFINIAK, ahrefs, Poland

Tensor shape mismatches are among the most common bugs in deep learning code. We present the shape inference system of OCANNL, an OCaml deep learning framework that combines *generalized einsum notation* with *row variables* and *constraint-based inference*. Our einsum notation extends the standard contraction syntax with strided iteration (for convolutions), axis concatenation (for block tensors), and row variables (for flexible axis counts). Shapes are organized into three axis kinds—batch, input, and output—reflecting deep learning semantics. A multi-stage constraint solver resolves dimension and row variables, producing informative error messages via provenance tracking. We integrate this system into OCaml through PPX syntax extensions, providing an ergonomic DSL that leverages OCaml's type and module systems. We illustrate the approach with multi-head attention, convolutions, and transformer blocks drawn from working OCANNL code.

1 Introduction

Deep learning frameworks must reconcile two competing demands: concise notation for tensor operations, and early detection of shape errors. Existing approaches each sacrifice one for the other:

- **Standard einsum** (NumPy, PyTorch) provides compact contraction notation but is limited to summation over shared indices. Convolutions, strided access, and concatenation require separate APIs. Shapes are checked at runtime.
- **Named tensors** (PyTorch named tensors [1], torchdim [2]) attach semantic labels to axes, but labels are verbose in einsum-style notation and create inference ambiguity when multiple axes share the same semantic unit.
- **Dependent types** (Dex [3], Futhark [4]) offer compile-time guarantees but demand heavy type annotations.
- **einops** [5] provides readable reshaping syntax but operates at the level of shape transformations rather than computations.

OCANNL[1] takes a different approach. Its shape system combines three ideas:

(1) A *generalized einsum notation* that unifies contraction, strided iteration, convolution, and concatenation in a single syntax.
(2) *Row variables*, borrowed from row polymorphism in type theory, that allow specifications without committing to exact axis counts.
(3) *Constraint-based inference* with a multi-stage solver that resolves shapes at model construction time, before any data flows through the network.

These are integrated into OCaml via PPX syntax extensions (`%op` and `%cd`), yielding an embedded DSL that feels native while providing shape safety. This paper describes the design, the inference algorithm, and its application to standard deep learning patterns.

2 The Shape System

2.1 Three Axis Kinds

Every tensor shape in OCANNL is partitioned into three rows of axes:

$$\underbrace{\text{batch}}_{\text{data samples}} \;\Big|\; \underbrace{\text{input}}_{\text{consumed}} \rightarrow \underbrace{\text{output}}_{\text{produced}}$$

[1]https://github.com/ahrefs/ocannl

Author's Contact Information: Lukasz Stafiniak, ahrefs, Poland, lukstafi@gmail.com.

This partition reflects deep learning semantics:

- **Batch axes** index independent data samples and are preserved across all operations.
- **Output axes** represent produced values (leftmost in conventional matrix notation).
- **Input axes** represent consumed values (rightmost), analogous to function arguments.

The input–output distinction gives tensor multiplication (*) the semantics of function composition: for tensors with shapes $|i \to o_1$ and $|i' \to o_2$, multiplication contracts i against o_2, producing $|i' \to o_1$—matching matrix multiplication when each row has a single axis. This convention extends naturally to higher-order tensors without requiring explicit einsum specs for common operations.

Each row is independently broadcastable: a dimension-1 axis broadcasts to match any dimension, and a row with fewer axes is extended with dimension-1 axes. Unlike NumPy, which pads from the left, OCANNL broadcasts "in the middle"—preserving both specified leading and trailing axes of a row.

2.2 Generalized Einsum Notation

OCANNL's einsum notation extends the standard contraction syntax in several directions. The general form for binary operations is:

```
rhs1_spec ; rhs2_spec => lhs_spec
```

where each spec describes the batch, input, and output axes of the corresponding tensor using the `batch | input -> output` convention. Unary operations use `rhs_spec => lhs_spec`. Within each spec, axes are identified positionally by single-character or multi-character labels.[2]

Standard contraction. Matrix multiplication is written as:

```
a +* "m n; n p => m p" b
```

The `+*` operator performs addition-reduction with multiplication (the standard einsum semantics). Labels appearing in both operand specs but not in the result spec are contracted.

Strided iteration and convolutions. Affine index expressions in specs support strided access:

```
x +* "... | stride*oh+kh, stride*ow+kw, ic;
     kh, kw, ic -> oc
     => ... | oh, ow, oc" ["kh"; "kw"] { kernel }
```

The expression `stride*oh+kh` means the input index is $stride \times oh + kh$, where `oh` is the output position and `kh` is the kernel position. Adding a dilation factor gives the standard convolution formula: $stride \times o + dilation \times k$. The shape inference system automatically derives the output dimensions from the input dimensions, stride, dilation, and kernel size via the relationship:

$$d_{\text{in}} = stride \times (d_{\text{out}} - 1) + 1 + (d_{\text{kernel}} - 1) \times dilation$$

Concatenation. The ˆ operator creates a concatenated axis:

```
(* Concatenate two vectors *)
(a, b) ++^ "x; y => x^y"
(* Extract a prefix *)
x ++^ "a^b => a"
```

[2]Multi-character mode is triggered by the presence of a comma: `"abc"` denotes three axes `a`, `b`, `c`, while `"abc,"` denotes a single axis named `abc`.

A concatenated axis `a^b` has dimension $d_a + d_b$ and iterates first over the range of `a`, then over the range of `b`. This enables block tensor construction and axis slicing within the same notation.

Dimension capture. Einsum specs can capture dimension variables for use in subsequent computations:

```
let scores =
  q +* "... s | h d; ... t | h d
        => ... s | t -> h" ["h"; "d"] k
in
Shape.set_dim h num_heads;
scores /. sqrt (dim d)
```

The list `["h"; "d"]` binds OCaml variables `h` and `d` to the inferred dimensions of the corresponding axes. These can be used to set dimension constraints (`Shape.set_dim`) or to compute scaling factors (`dim d` retrieves the integer dimension value).

Additional operators. Beyond the standard addition-multiply einsum (`+*`), OCANNL provides: `@^+` for tropical semiring operations (max-reduce with addition), `++` for unary add-reduction, and `@^^` for unary max-reduction. These share the same spec syntax, differing only in the accumulation and combination operations.

2.3 Row Variables

Row variables are the key mechanism for writing shape-polymorphic specifications. Borrowed from row polymorphism in type theory [6], they represent an unknown sequence of axes.

- `...` is a context-dependent ellipsis that introduces an anonymous row variable for the enclosing axis kind.
- `..name..` (or `..n..` in single-character mode) is a named row variable. Occurrences of the same name within a spec denote the same row variable and must unify to the same sequence of axes.

Row variables implement the "principle of least commitment": a specification should constrain only the axes it cares about. For example, the softmax spec:

```
x ++ " ... | t -> ... => ... | 0 -> ..."
```

reduces over the output axis named `t` while preserving all batch axes, all other output axes, and all input axes—regardless of how many there are. Without row variables, this would require separate implementations for each possible axis count.

Row variables in OCANNL interact with broadcasting: a row variable can unify with zero or more axes, and dim-1 axes introduced by broadcasting count as legitimate axes. The inference system resolves row variables progressively: first unifying what is determined by constraints, then closing remaining variables to their least upper bounds or to "no further axes" at later stages.

2.4 Dimension Units

Axes have optional semantic annotations called *dimension units* (termed "basis" in the codebase). Unlike named tensors, where axes are identified by their names, OCANNL axes are identified *positionally* and may optionally carry a unit that constrains matching: axes with different units cannot be unified.

This design avoids two problems with named axes:

(1) **Verbosity**: named axes require explicit labels on every axis of every tensor. Positional identification builds on mathematical tradition (linear algebra, Einstein notation) where position carries meaning.
(2) **Inference ambiguity**: when multiple axes share the same semantic unit (e.g., two spatial dimensions both representing "position"), named-axis systems cannot determine which name goes where. Positional identification with optional units avoids this entirely.
(3) **Row variable compatibility**: row variables represent unknown numbers of axes. Named-axis systems cannot express "some unknown axes with unknown names" without reverting to positional structure. OCANNL's positional axes compose naturally with row variables.
(4) **Conciseness**: the same matrix multiplication is `"i,j;j,k=>i,k"` in OCANNL versus `(A[i,k]*B[k,j]).` in torchdim—positional einsum is more compact for contraction-heavy patterns.

For example, a unit `"rgb"` attached to a dimension of size 3 denotes that an axis represents color channels. Shape inference ensures that axes unified with this one also have unit `"rgb"` and size 3, providing semantic safety. But the axis is still identified by its position in the batch/input/output partition, not by its name.

3 Constraint-Based Shape Inference

3.1 Representing Constraints

Shape inference maintains a global environment mapping dimension variables and row variables to their current knowledge:

```
dim_var ↦ Solved(dim) | Bounds(lub, constraints, ordering)
row_var ↦ Solved(row) | Bounds(lub, constraints, ordering)
```

The constraint language includes:

- **Equations**: $d_1 = d_2$ (dimension equality), $r_1 = r_2$ (row equality).
- **Inequalities**: $d_c \geq d_s$ (where d_c is from a super-tensor and d_s from a sub-tensor, reflecting broadcasting), $r_c \geq r_s$ (row inequality).
- **Dimension constraints**: minimum dimension, total element count.
- **Row constraints**: total element count (possibly involving strided dimension variables), exact axis lists.

For ground dimensions, $n \geq m$ means either $n = m$ or $m = 1$ (broadcasting). For ground rows, $q \geq r$ means q has at least as many axes as r, and for each corresponding pair of dimensions, q's dimension is $\geq r$'s. This combines row polymorphism with nominal subtyping: the subtyping relation states that dimension-1 (no unit) is smaller than all dimensions.

Einsum operations generate *equations* (not inequalities), so they do not permit broadcasting—they require exact shape matching. Pointwise and compose operations generate inequalities, allowing broadcasting.

3.2 The Multi-Stage Solver

Constraints are solved in stages, progressing from conservative (online, as tensors are composed) to aggressive (on demand, when dimensions or projections are needed):

(1) **Stage 1 (online)**: Unification and constraint propagation as tensor expressions are constructed. This resolves what can be determined immediately.
(2) **Stage 2**: Force precision-related coefficients (byte sizes of numeric types).
(3) **Stage 3**: Close row variables in *terminal* (leaf) shapes to their least upper bounds (LUBs). Terminal shapes are closed first because their shapes can only be determined by downstream

constraints (they can always be broadcast), unlike composite shapes which are forced "from below" by their components.

(4) **Stage 4**: Close terminal dimension variables with multiple lower bounds to their LUBs. Resolve `Total_elems` constraints where row variables have been eliminated.
(5) **Stage 5**: Guess remaining terminal dimension variables to 1 (the broadcasting identity), unless prevented by `Total_elems` constraints. Resolve remaining row constraints heuristically.
(6) **Stage 6**: Extend LUB processing to non-terminal shapes. Set remaining row variables to "no further axes."
(7) **Stage 7**: Close all remaining dimension variables to their lower bounds or to 1.

The terminal-first strategy (Stages 3–5 before 6–7) reflects an asymmetry: leaf tensors acquire shape information only from downstream constraints and broadcasting, so they must be resolved first. Composite tensors then get their shapes forced by their already-resolved components.

Preventing premature guessing. A subtle issue arises with strided dimensions (convolutions): if a dimension variable appears in the numerator of a `Total_elems` constraint (e.g., total = coeff $\times$ d/denom), guessing $d = 1$ prematurely can make the constraint unsatisfiable. The solver tracks these dependencies via a `has_uniq_constr_unless` field: a numerator variable cannot be guessed to 1 unless at least one denominator variable is also prevented from guessing (breaking potential cycles).

3.3 Error Provenance

Each row carries a *provenance* record tracking which einsum spec, tensor expression, or operation produced it. When unification fails—e.g., two axes have incompatible dimensions or units—the error message includes the provenance chain, allowing the user to trace the conflict back to the source-level einsum specification.

4 OCaml Integration

OCANNL integrates with OCaml via two PPX syntax extensions:

- `%op`: for tensor expressions with automatic differentiation. Records like `{ w }` declare learnable parameters.
- `%cd`: for assignment computations (code generation without gradient tracking).

Both extensions transform OCaml expressions, reinterpreting arithmetic operators as tensor operations and record syntax as tensor declarations. A unit parameter `()` separates configuration arguments (processed as plain OCaml) from tensor arguments (transformed by the extension):

```
let%op mlp_layer ~label ~hid_dim () x =
  relu (({ w } * x) + { b = 0.; o = [hid_dim] })
```

The record `{ w }` declares a parameter with default initialization; `{ b = 0.; o = [hid_dim] }` declares a bias initialized to zero with explicit output dimensions. The `[hid_dim]` syntax specifies a one-dimensional output of size `hid_dim`. The `%oc` anti-quotation escapes back to plain OCaml when needed inside a transformed expression.

This design leverages OCaml's module system: the extensions require specific DSL modules in scope (`TDSL` for `%op`, `NTDSL` for `%cd`), which determine gradient tracking behavior. The modules are brought into scope uniformly via `open Ocannl.Operation.DSL_modules`.

5 Examples

5.1 Multi-Head Attention

The multi-head attention mechanism illustrates several features working together. In OCANNL:

```
let%op multi_head_attention ~label ~num_heads
    ~d_k ~d_v ?temperature
    ?(dropout_rate = 0.0) ()
    ~train_step ?mask x =
  let q = { w_q } * x in
  let k = { w_k } * x in
  let v = { w_v } * x in
  let scores =
    (q +* k " ... s | h d; ... t | h d
            => ... s | t -> h" ["h"; "d"])
    /. sqrt (dim d)
  in
  Shape.set_dim h num_heads;
  Shape.set_dim d d_k;
  Shape.set_dim e d_v;
  let attn_weights =
    softmax ~spec:" ... | t -> ..." ?temperature ()
      (match mask with
       | None -> scores
       | Some mask -> where mask scores !.(-1e9))
  in
  let attn_weights =
    dropout ~rate:dropout_rate ()
      ~train_step attn_weights
  in
  { w_o } *
    (attn_weights +* v
       " ... s | t -> h; ... t | h e
         => ... s | h e" ["e"])
```

Key observations:

- **Row variables** (...): the batch dimensions are handled polymorphically—the same code works for any number of batch axes.
- **Input–output split**: in the score computation, `... s | t -> h` makes `t` (time/key positions) an output axis and `h` (heads) an input axis. This means the softmax reduces over `t` while keeping `h` independent—exactly the attention semantics.
- **Dimension capture**: `["h"; "d"]` captures the head and key dimensions for scaling (`sqrt (dim d)`) and constraint setting (`Shape.set_dim h num_heads`).
- **Implicit shapes**: the projection matrices `{w_q}`, `{w_k}`, `{w_v}`, `{w_o}` have their shapes entirely inferred from context—no dimension annotations needed.

5.2 Convolution

A 2D convolution with configurable stride and kernel size:

```
let%op conv2d ?(stride=1) ?(kernel_size=3) () x =
  Shape.set_dim kh kernel_size;
  Shape.set_dim kw kernel_size;
```

Table 1. Attention score computation across frameworks.

Framework	Expression
NumPy	`np.einsum("bhsd,bhtd->bhst", q, k) / sqrt(d_k)`
PyTorch	`torch.matmul(q, k.transpose(-2,-1)) / math.sqrt(d_k)`
einops	(no native einsum; reshape + `torch.matmul`)
OCANNL	`(q +* k "...s|hd; ...t|hd => ...s|t->h" ["h";"d"]) /. sqrt (dim d)`

```
x +* "... | stride*oh+kh, stride*ow+kw, ic;
      kh, kw, ic -> oc
      => ... | oh, ow, oc" ["kh"; "kw"] { kernel }
+ { bias = 0. }
```

The spec `stride*oh+kh` expresses that input position = $stride \times oh + kh$. The inference system derives the output spatial dimensions from the input dimensions, stride, and kernel size. Row variables (...) handle arbitrary batch dimensions.

5.3 Layer Normalization

Layer normalization demonstrates dimension capture and reduction:

```
let%op layer_norm ~label ?(epsilon = 1e-5) () x =
  let mean =
    x ++ " ... | ..d.. => ... | 0 " ["d"] in
  let centered = (x - mean) /. dim d in
  let variance =
    (centered *. centered)
      ++ " ... | ... => ... | 0 " in
  let std_dev = sqrt (variance + !.epsilon) in
  let normalized = centered /. std_dev in
  ({ gamma = 1. } *. normalized) + { beta = 0. }
```

The spec `... | ..d.. => ... | 0` reduces *all* output axes to a scalar (0 denotes a removed axis), while the row variable `..d..` captures the total output dimensionality for normalization. The named row variable means that however many output axes there are, they are all reduced and the product of their sizes is accessible via `dim d`.

5.4 Comparison with Other Frameworks

Table 1 compares how a dot-product attention score computation is expressed across frameworks.

Key differences:

- NumPy einsum requires the user to commit to exact axis counts (`bhsd`)—adding a batch dimension requires changing the spec. OCANNL's ... handles this polymorphically.
- PyTorch uses index-based transposition, obscuring the operation's intent. OCANNL's spec makes the contraction pattern explicit.

- The OCANNL version captures `d` for scaling and `h` for setting the number of heads, avoiding separate bookkeeping.
- The input/output distinction (`t -> h` in the result) carries semantic meaning: `t` is an output axis (produced by the operation), while `h` is an input axis (preserved for the subsequent value-weighting step).

6 Related Work

Einsum extensions. The original einsum notation dates to NumPy [7] and has been adopted by TensorFlow and PyTorch. `opt_einsum` [8] optimizes contraction order but does not extend the notation. einops [5] extends einsum with reshape and repeat operations but does not support convolutions or constraint-based inference. OCANNL's generalized einsum subsumes standard contraction and adds strided iteration, convolution, and concatenation.

Named tensors and axis labels. Three approaches to named dimensions have been explored. Named tensor notation [1] formalizes attaching permanent string labels to axes, enabling type-like checking of dimension alignment. DumPy [10] applies temporary string labels at operation sites and compiles to `jax.vmap`, achieving near-loop clarity without implicit broadcasting—external labels are discarded after each operation, validating OCANNL's approach of using temporary pseudo-labels in einsum specifications. Torchdim [11] introduces first-class dimension objects with three rules [11]: implicit batching over the union of dimensions, dimensions as dimension specifiers, and dimensions as index-range tensors. The third rule enables index-component operations (e.g., upper-triangular masks) that are complementary to einsum-based computation and could be adopted orthogonally. Torchdim has been upstreamed into PyTorch as `functorch.dim` [2].

All three named-axis approaches face inference ambiguity when multiple axes share the same semantic unit. OCANNL's positional identification with optional units avoids this while row variables provide the flexibility that named approaches achieve through batching rules.

Dependent types for arrays. Dex [3] uses dependent types with index sets to ensure shape safety. Futhark [4] uses size types. Both provide strong static guarantees but require explicit type annotations that OCANNL's inference makes unnecessary. OCANNL's approach is closer in spirit to Hindley-Milner inference: shapes are inferred from usage rather than declared.

Row polymorphism. Row polymorphism was introduced by Rémy [6] for record types. It has been applied to effect systems [9] and extensible records. To our knowledge, OCANNL is the first system to apply row polymorphism to tensor shape inference, using row variables to represent unknown sequences of axes.

7 Discussion and Future Work

Limitations. OCANNL's shape system is currently monomorphic: variables are interpreted existentially, not universally. A tensor-producing function that is called twice shares shape variables between calls, which can over-constrain shapes. The workaround is the unit-parameter pattern (`let layer = make () in ...`), which creates fresh variables per call. True shape polymorphism would require abstracting over fresh variables when forming tensor functions and freshening them at application sites—essentially an abstract interpretation mechanism. This is planned for a future version.

Shape schemes. A natural extension would be *shape schemes*: universally quantified shape specifications that are freshened at each use site, analogous to type schemes in Hindley-Milner. This would eliminate the need for unit-parameter workarounds and enable shape-polymorphic libraries.

Formal guarantees. The current system relies on testing rather than formal proofs for correctness. Formalizing the constraint language and proving soundness of the multi-stage solver would strengthen confidence, particularly around the interaction of row variables with broadcasting and strided dimensions.

8 Conclusion

We have presented OCANNL's shape inference system, which combines generalized einsum notation, row variables, and constraint-based inference to provide shape safety for deep learning code. The three axis kinds (batch/input/output) capture deep learning semantics naturally. Row variables enable the "principle of least commitment"—specifications constrain only the axes that matter. The multi-stage solver resolves shapes before execution, with provenance-tracked error messages. Integration via OCaml PPX extensions provides an ergonomic surface syntax. Our examples demonstrate that this approach handles standard deep learning patterns (attention, convolutions, normalization) concisely and safely.

OCANNL is open source and available at https://github.com/ahrefs/ocannl.

References

[1] David Chiang, Alexander M. Rush, and Sasha Rush. Named tensor notation. In *NeurIPS*, 2021.
[2] Jason Ansel et al. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *ASPLOS*, 2024.
[3] Adam Paszke, Daniel D. Johnson, David Duvenaud, Dimitrios Vytiniotis, Alexey Radul, Matthew J. Johnson, Jonathan Ragan-Kelley, and Dougal Maclaurin. Getting to the point: Index sets and parallelism-preserving autodiff for pointful array computation. *Proc. ACM Program. Lang.*, 5(ICFP), 2021.
[4] Troels Henriksen, Niels G. W. Serup, Martin Elsman, Fritz Henglein, and Cosmin E. Oancea. Futhark: Purely functional GPU-programming with nested parallelism and in-place array updates. In *PLDI*, 2017.
[5] Alex Rogozhnikov. Einops: Clear and reliable tensor manipulations with Einstein-like notation. In *ICLR*, 2022.
[6] Didier Rémy. Type inference for records in a natural extension of ML. In Carl A. Gunter and John C. Mitchell, editors, *Theoretical Aspects of Object-Oriented Programming*, pages 67–95. MIT Press, 1993.
[7] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
[8] Daniel G. A. Smith and Johnnie Gray. opt_einsum: A Python package for optimizing contraction order for einsum-like expressions. *JOSS*, 3(26):753, 2018.
[9] Daan Leijen. Koka: Programming with row polymorphic effect types. *MSFP*, 153:100–126, 2014.
[10] Dynomight. DumPy: NumPy except it's OK if you're dum. https://dynomight.net/dumpy/, 2025.
[11] Zachary DeVito. Torchdim: First-class dimensions for PyTorch. https://github.com/facebookresearch/torchdim, 2024. Archived; upstreamed into PyTorch as `functorch.dim`.